

ML Assignment-4

Pola Venkata Revanth (2022CS51650), G Anudeep (2022CS11617)

November 25, 2025

1 Introduction

This assignment involves implementing and analyzing two neural architectures for sequence prediction: a Recurrent Neural Network (RNN) with Bahdanau attention, and a Transformer encoder-decoder model.

2 Model Architecture

Encoder-Decoder with Bahdanau Attention

Let the input source sentence be:

$$X = (x_1, x_2, \dots, x_{T_x})$$

Each token is embedded:

$$x_t^{emb} = E_s(x_t), \quad x_t^{emb} \in \mathbb{R}^{128}$$

Encoder

We use a stacked (2-layer) unidirectional RNN with hidden size $H = 512$. Let $x_t^{emb} \in \mathbb{R}^{128}$ be the embedding of the input token at time t .

The stacked RNN computes layer-wise hidden states for each token:

$$\begin{aligned} h_t^{(1)} &= \text{RNN}^{(1)}(x_t^{emb}, h_{t-1}^{(1)}), & h_t^{(1)} &\in \mathbb{R}^H, \\ h_t^{(2)} &= \text{RNN}^{(2)}(h_t^{(1)}, h_{t-1}^{(2)}), & h_t^{(2)} &\in \mathbb{R}^H. \end{aligned}$$

For a source sequence of length T_x the encoder returns:

$$\mathbf{outputs} = [h_1^{(2)}, h_2^{(2)}, \dots, h_{T_x}^{(2)}] \in \mathbb{R}^{T_x \times H},$$

and the final hidden states of all layers:

$$\mathbf{hidden} = \begin{bmatrix} h_{T_x}^{(1)} \\ h_{T_x}^{(2)} \end{bmatrix} \in \mathbb{R}^{2 \times H}.$$

When batching B sequences (padded to the same length) these shapes become:

$$\mathbf{outputs} \in \mathbb{R}^{B \times T_x \times H}, \quad \mathbf{hidden} \in \mathbb{R}^{L \times B \times H},$$

where $L = 2$ is the number of RNN layers.

Bahdanau Additive Attention

At decoder step i , the score is computed as:

$$e_{i,t} = v^\top \tanh(W_s s_{i-1} + W_h \tilde{h}_t)$$

where parameter dimensions are:

$$s_{i-1}, \tilde{h}_t \in \mathbb{R}^{512}, \quad W_s, W_h \in \mathbb{R}^{256 \times 512}, \quad v \in \mathbb{R}^{256}$$

Attention weights:

$$\alpha_{i,t} = \frac{\exp(e_{i,t})}{\sum_{k=1}^{T_x} \exp(e_{i,k})}$$

Context vector:

$$c_i = \sum_{t=1}^{T_x} \alpha_{i,t} \tilde{h}_t, \quad c_i \in \mathbb{R}^{512}$$

Decoder

The decoder generates the output sequence one token at a time. At decoding step t , it receives: (i) previous output token, (ii) previous hidden state, and (iii) encoder hidden states (for attention).

1) Input Embedding The previously generated token y_{t-1} is converted into a dense vector:

$$x_{t-1}^{emb} = \text{Embed}(y_{t-1}), \quad x_{t-1}^{emb} \in \mathbb{R}^E$$

2) Bahdanau Additive Attention The top layer decoder hidden state from the previous timestep

$$s_{t-1}^{(top)} \in \mathbb{R}^H$$

is compared against each encoder output $h_i \in \mathbb{R}^H$ to compute alignment scores:

$$e_{t,i} = \mathbf{v}^\top \tanh(\mathbf{W}_a s_{t-1}^{(top)} + \mathbf{U}_a h_i)$$

Attention weights:

$$\alpha_{t,i} = \text{softmax}(e_{t,i})$$

Context vector:

$$c_t = \sum_{i=1}^S \alpha_{t,i} h_i, \quad c_t \in \mathbb{R}^H$$

3) RNN Update (2-layer RNN) The decoder updates its hidden state using both the embedded token and attention context:

$$s_t = \text{RNN}([x_{t-1}^{emb}; c_t], s_{t-1}), \quad s_t^{(top)} \in \mathbb{R}^H$$

4) Output Prediction Decoder state and context are merged and projected to vocabulary dimension:

$$\begin{aligned} o_t &= \tanh(\mathbf{W}_c [c_t; s_t^{(top)}]), & o_t &\in \mathbb{R}^H \\ \hat{y}_t &= \text{softmax}(\mathbf{W}_o o_t), & \hat{y}_t &\in \mathbb{R}^V \end{aligned}$$

Dimensions Used in Our Model

$$E = 128, \quad H = 512, \quad V = \text{vocab size}, \quad \text{num_layers} = 2$$

Summary: Attention focuses on relevant encoder states at each step, while the decoder RNN learns temporal structure of the target path.

Seq2Seq Training with Teacher Forcing and Maze Conditioning

The encoder produces:

$$\mathbf{outputs}^{enc} \in \mathbb{R}^{B \times T_x \times H}, \quad \mathbf{hidden}^{enc} \in \mathbb{R}^{L \times B \times H}$$

We embed the maze type and add it only to the top encoder layer:

$$m = \text{Embed}(\text{maze_type}), \quad m' = \mathbf{W}_m m$$

$$h_{T_x}^{(L)} \leftarrow h_{T_x}^{(L)} + m'$$

The decoder starts with the last non-PAD input token (i.e., `<PATH_START>`) and the encoder’s final hidden state:

$$\tilde{y}_0 = \langle \text{PATH_START} \rangle, \quad s_0 = \text{hidden}^{enc}$$

At each timestep t , the decoder predicts the next token:

$$\hat{y}_t = \text{Decoder}(s_{t-1}, \tilde{y}_{t-1}, \mathbf{outputs}^{enc})$$

Teacher forcing is used with probability p_{tf} :

$$\tilde{y}_{t-1} = \begin{cases} y_{t-1}, & \text{with probability } p_{tf} \\ \hat{y}_{t-1}, & \text{otherwise} \end{cases}$$

The model is trained by minimizing cross-entropy loss:

$$\mathcal{L} = - \sum_{t=1}^{T_y} \log P(y_t \mid y_{<t}, X)$$

RNN Results

Final metrics after 20 epochs (from training logs):

- **Train Loss:** 0.1700
- **Train Token Accuracy:** 0.9465
- **Test Token Accuracy:** 0.6900
- **Train Sequence Accuracy:** 0.7570
- **Test Sequence Accuracy:** 0.6465
- **Test Micro-F1 Score:** 0.6900

Additionally, when evaluated with **no teacher forcing**, the model achieves a test sequence accuracy of **0.548**.

```
Epoch 20/20
Training: 100% |████████████████████████████████████████████████████████████████████████████████| 2500/2500 [03:44<00:00, 11.14it/s]
Evaluating: 100% |████████████████████████████████████████████████████████████████████████████████| 625/625 [00:24<00:00, 25.11it/s]
Train Loss: 0.1700 | Test Loss: 1.8267 | Train Tok Acc: 0.9465 | Test Tok Acc: 0.6900 | Train Seq Acc: 0.7570 | Test Seq Acc: 0.6465 | Test f1 Score: 0.6900
```

RNN: Loss vs Epochs

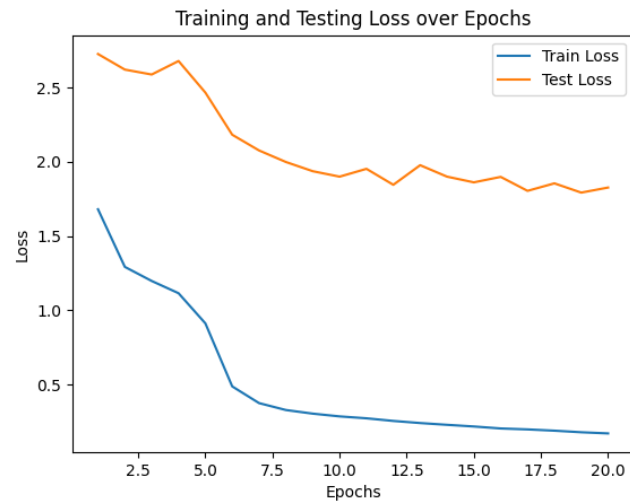


Figure 1: RNN training and test loss over epochs.

RNN: Token Accuracy

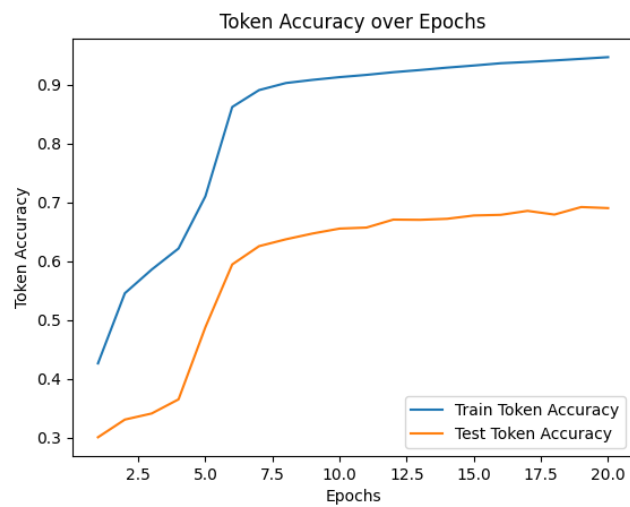


Figure 2: RNN train and test token accuracy.

RNN: Sequence Accuracy

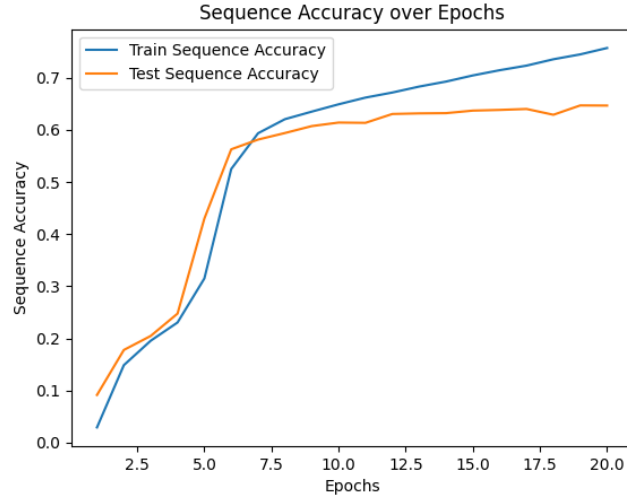


Figure 3: RNN train and test sequence accuracy.

3 Transformer Architecture

The model is a standard Encoder–Decoder Transformer constructed using PyTorch modules. All computations below directly match the forward pass implemented in our code.

1. Embedding and Positional Encoding

Input token sequence:

$$X = (x_1, x_2, \dots, x_{T_x}), \quad X \in \mathbb{Z}^{B \times T_x}.$$

Token embedding:

$$E = \text{Embed}(X) \in \mathbb{R}^{B \times T_x \times 128}.$$

Sinusoidal positional encoding:

$$Z = E + PE, \quad Z \in \mathbb{R}^{B \times T_x \times 128}.$$

2. Encoder

The encoder contains 6 stacked layers of

$$\text{TransformerEncoderLayer}(d_{\text{model}} = 128, n_{\text{heads}} = 8, \text{ff} = 512).$$

Encoder output:

$$M = \text{Encoder}(Z), \quad M \in \mathbb{R}^{B \times T_x \times 128}.$$

3. Decoder Input

Target sequence (shifted):

$$Y_{\text{in}} \in \mathbb{R}^{B \times T_y}.$$

Embedded + positional encoded:

$$T = \text{Embed}(Y_{\text{in}}) + PE, \quad T \in \mathbb{R}^{B \times T_y \times 128}.$$

Causal mask:

$$\text{Mask}[i, j] = \begin{cases} -\infty & j > i, \\ 0 & j \leq i. \end{cases}$$

4. Decoder

The decoder contains 6 layers of

$$\text{TransformerDecoderLayer}(128, 8, 512).$$

Forward pass:

$$D = \text{Decoder}(T, M), \quad D \in \mathbb{R}^{B \times T_y \times 128}.$$

5. Output Projection

Vocabulary logits:

$$\begin{aligned} O_t &= W_o D_t, \quad W_o \in \mathbb{R}^{V \times 128}, \\ O &\in \mathbb{R}^{B \times T_y \times V}. \end{aligned}$$

6. Training Objective

Shifted target:

$$Y_{\text{out}} \in \mathbb{R}^{B \times T_y}.$$

Cross-entropy loss with padding ignored:

$$\mathcal{L} = - \sum_{t=1}^{T_y} \log P(Y_{\text{out},t} \mid X, Y_{\text{in}, < t}).$$

7. Inference

Initialize:

$$y_1 = \langle \text{PATH_START} \rangle.$$

Iteratively:

$$y_t = \arg \max O_{t-1},$$

until $\langle \text{PATH_END} \rangle$ is produced.

Summary

$$X \xrightarrow{\text{Embed+PE}} Z \xrightarrow{\text{Encoder}} M$$

$$Y_{\text{in}} \xrightarrow{\text{Embed+PE}} T \xrightarrow{\text{Decoder}(M)} D \xrightarrow{W_o} O$$

All tensor shapes match the implementation:

$$Z, M \in \mathbb{R}^{B \times T_x \times 128}, \quad T, D \in \mathbb{R}^{B \times T_y \times 128}, \quad O \in \mathbb{R}^{B \times T_y \times V}.$$

Transformer Results

Final metrics after 20 epochs (from training logs):

- **Train Loss:** 0.2566
- **Test Token Accuracy:** 87.89%
- **Train Sequence Accuracy:** 88.36%
- **Test Sequence Accuracy:** 37.34%
- **Test Micro-F1 Score:** 87.89%

```
Epoch 20/20 train_loss=0.2566 test_loss=0.2590 train_seq_acc=88.36% test_tok_acc=87.89% test_seq_acc=37.34% test_f1=87.89%
```

Training Curves

Transformer: Loss vs Epochs

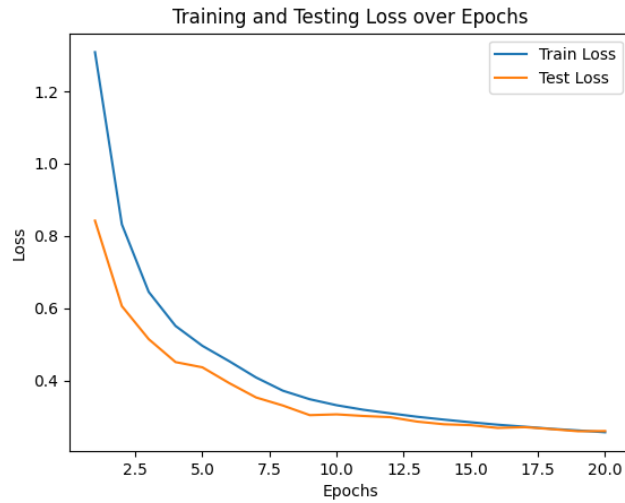


Figure 4: Transformer training and test loss over epochs.

Transformer: Token Accuracy

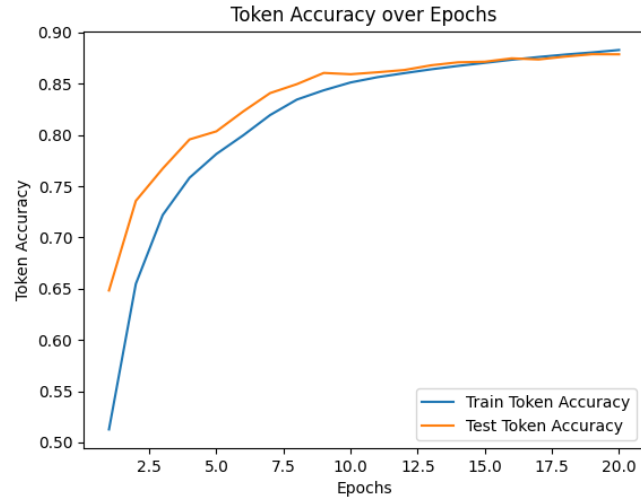


Figure 5: Transformer train and test token accuracy.

Transformer: Sequence Accuracy

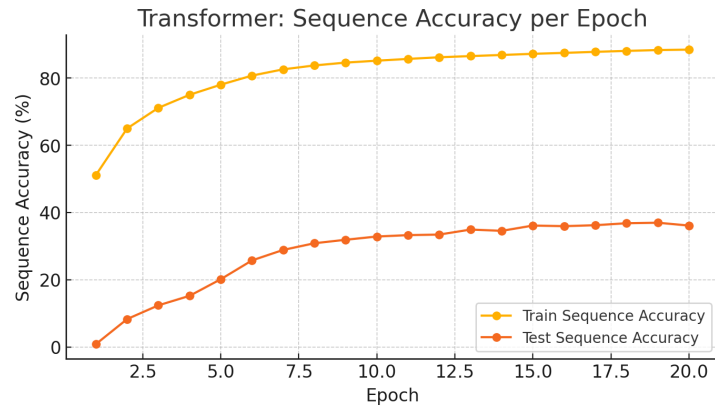


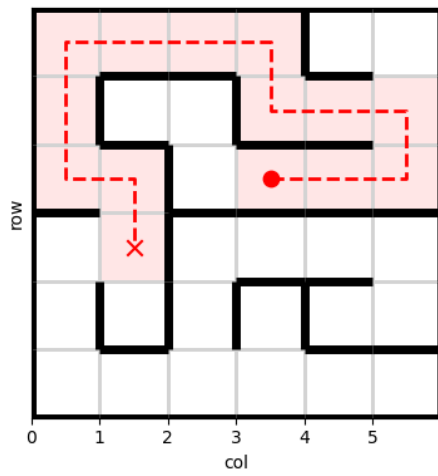
Figure 6: Transformer train and test sequence accuracy.

4 Qualitative Analysis: Predicted vs Ground-Truth Paths

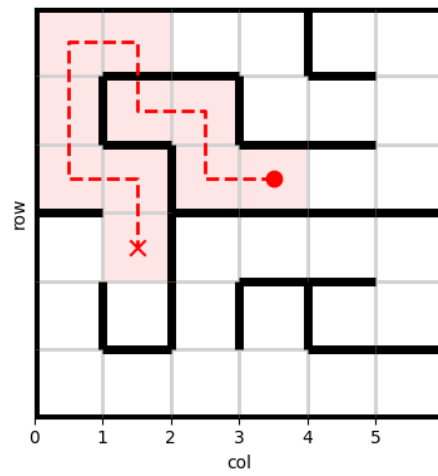
We present qualitative results for both models on 5 randomly sampled mazes from the validation set. Each example shows:

- the **Ground-Truth Path**, and
- the **Predicted Path** produced by the model.

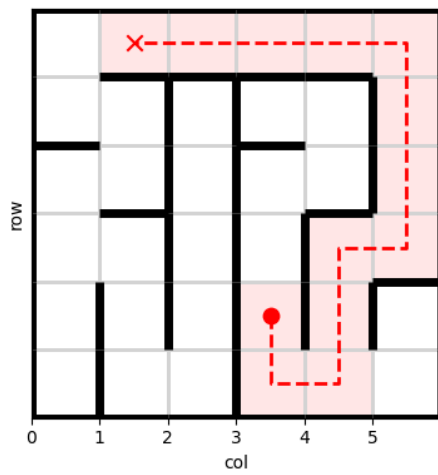
RNN with Bahdanau Attention



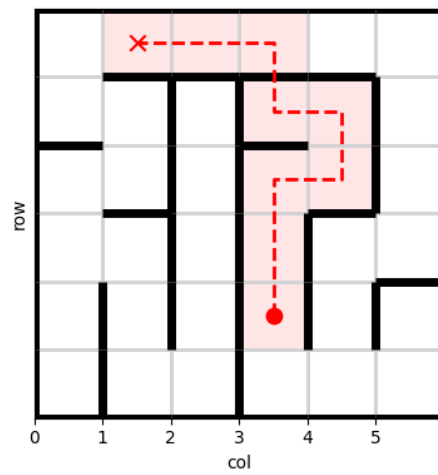
(a) Ground Truth (Example 0)



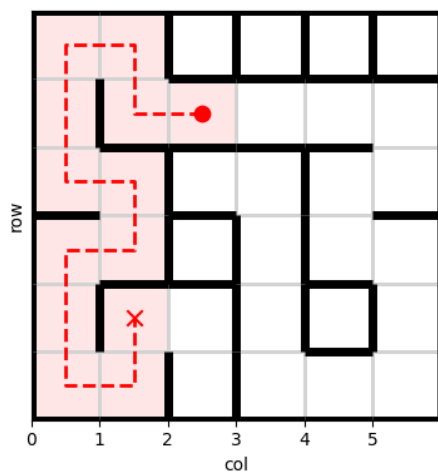
(b) Predicted Path (Example 0)



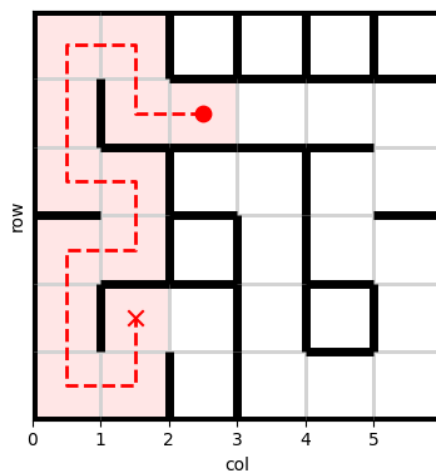
(a) Ground Truth (Example 1)



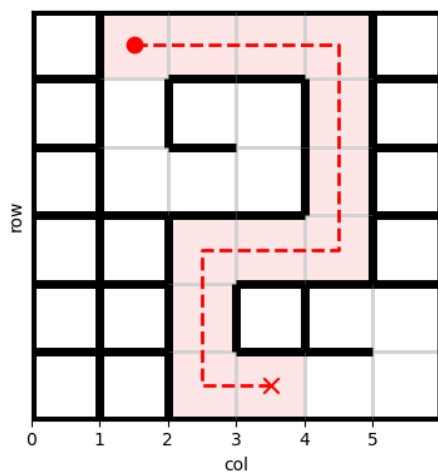
(b) Predicted Path (Example 1)



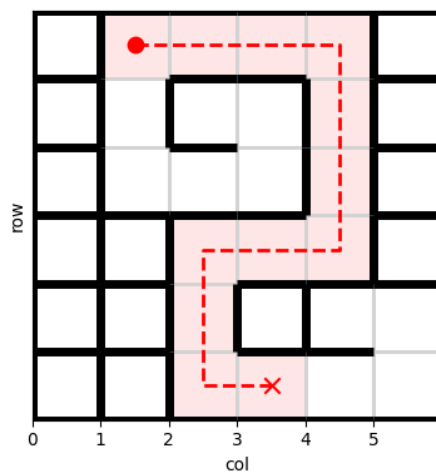
(a) Ground Truth (Example 2)



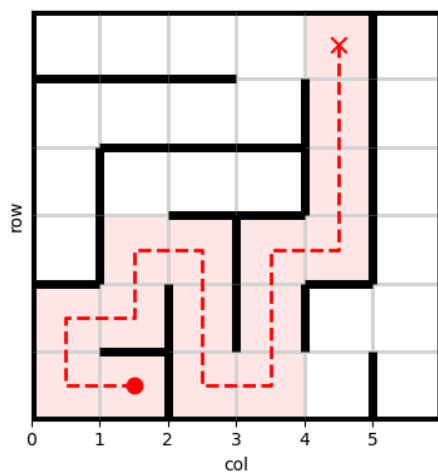
(b) Predicted Path (Example 2)



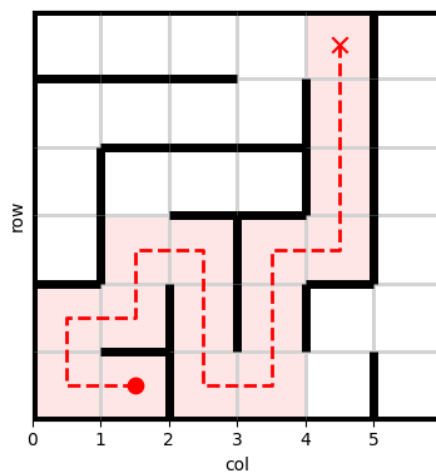
(a) Ground Truth (Example 3)



(b) Predicted Path (Example 3)

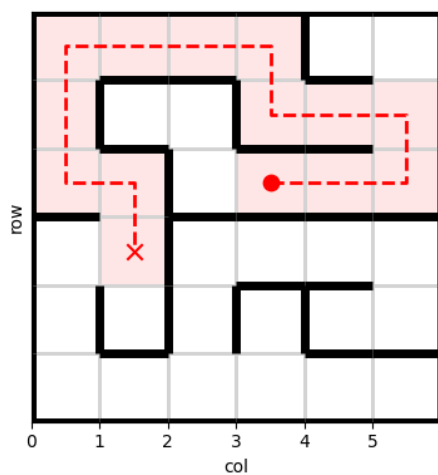


(a) Ground Truth (Example 4)

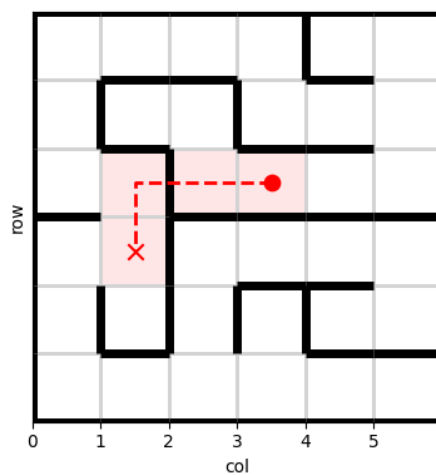


(b) Predicted Path (Example 4)

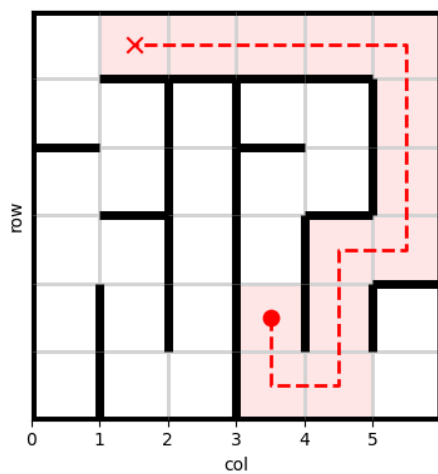
Transformer Model



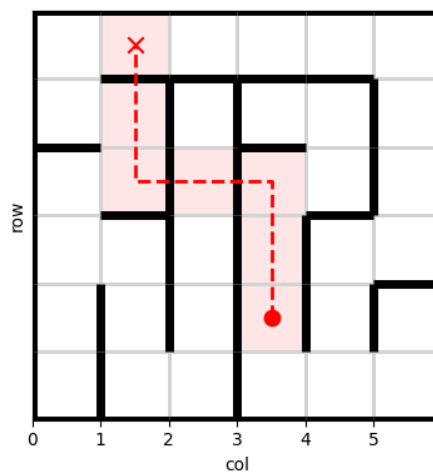
(a) Ground Truth (Example 0)



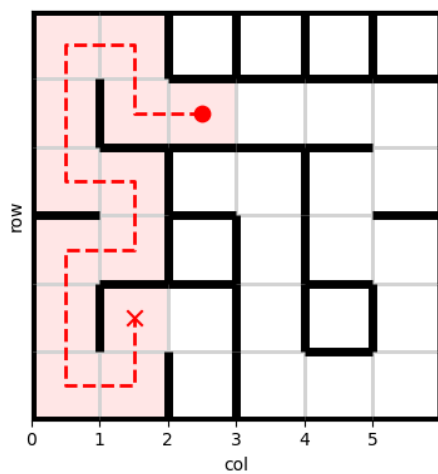
(b) Predicted Path (Example 0)



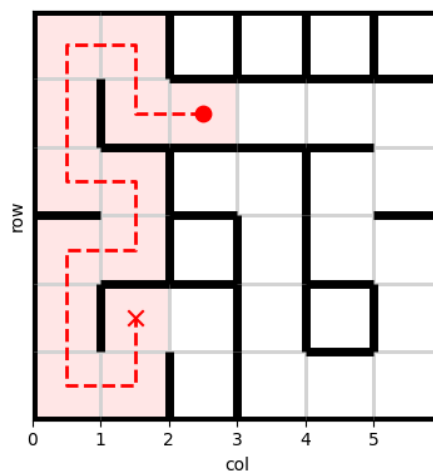
(a) Ground Truth (Example 1)



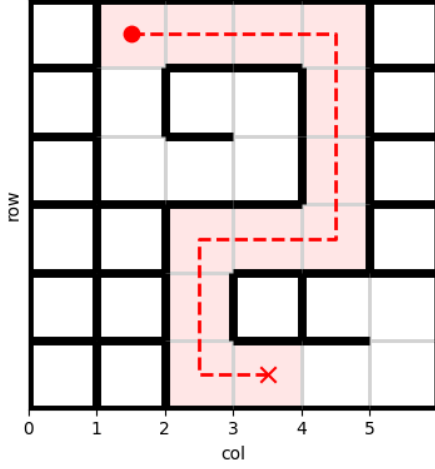
(b) Predicted Path (Example 1)



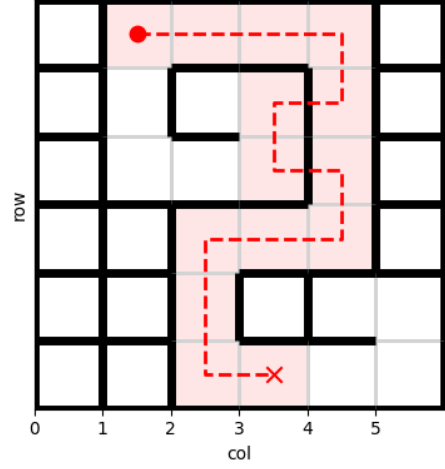
(a) Ground Truth (Example 2)



(b) Predicted Path (Example 2)



(a) Ground Truth (Example 3)



(b) Predicted Path (Example 3)

Why Transformer Sequence Accuracy is Lower than RNN

Although the Transformer achieves a high token accuracy, its sequence accuracy is significantly lower than the RNN model. This behaviour is expected and can be explained through the following reasons:

- **1. RNNs enforce strong temporal continuity.** The RNN decoder updates its hidden state recurrently:

$$s_t = f(s_{t-1}, y_{t-1}),$$

leading to smoother, more stable step-to-step predictions. This property is ideal for path-generation tasks where each step depends heavily on the previous one.

- **2. Transformers suffer more from exposure bias.** During training, the decoder always receives the ground-truth token (teacher forcing), but during inference it must rely on its own predictions. Small mistakes early in decoding propagate strongly and break the entire sequence.
- **3. Maze paths have strong sequential structure.** Path generation requires strict left-to-right dependency. RNNs handle such autoregressive tasks naturally, while Transformers—being parallel and attention-based—do not enforce such tight temporal order.

Summary: RNN recurrence stabilises autoregressive decoding and maintains sequence consistency, making it well-suited for maze path prediction. Transformers, despite higher token accuracy, are more sensitive to small prediction errors and therefore show lower sequence accuracy.